

SkillTree IRL

FULL PRODUCTION PRODUCT SPECIFICATION

Authoritative build specification for Codex

Product premise

Goals change. Skills persist. History persists. The user's character continuously develops.

Scope

Complete specification: product model, UX, goals, skills, XP, quests, habits, achievements, insights, social/sharing, imports/integrations, architecture, security, subscriptions, testing, deployment, operations and production-readiness requirements across sections 0-188.

This PDF is generated from the complete supplied Markdown specification; it is not a shortened summary.

Contents

Sections 0-188

1. Core Product Principles
2. Product Navigation
3. Mobile Navigation
4. Authentication
5. New User Onboarding
6. Initial Goal Selection
7. Suggested Goal Templates
8. Goal Setup Wizard
9. Goal Creation - Advanced Configuration
10. Goal States
11. Changing Goals
12. Goal Detail Page
13. Composite Goals
14. Goal Dependencies
15. Skills
16. Skill Taxonomy
17. Dynamic Skill Discovery
18. Skill Tree View
19. Skill Node
20. Skill Detail
21. XP System
22. XP Difficulty
23. XP Diminishing Repetition
24. Level Formula
25. Overall User Level
26. Activities
27. Activity Logging
28. Activity Evidence
29. Evidence Confidence
30. Quests
31. Quest States
32. Suggested Quests
33. Habits
34. Habit Tracking
35. Momentum
36. Streaks
37. Milestones
38. Achievements
39. Achievement Rarity
40. Today Screen
41. Next Best Action
42. Goal Reviews
43. History
95. Validation
96. Error Model
97. Optimistic UI
98. File Storage
99. Evidence Limits
100. Subscription Model
101. Subscription Billing
102. Entitlements
103. Trial
104. Account Export
105. Account Deletion
106. Privacy
107. Analytics
108. Product Metrics
109. User Feedback
110. Admin Platform
111. Feature Flags
112. Audit Logging
113. Rate Limiting
114. Abuse Controls
115. Security Headers
116. CSRF
117. XSS
118. Database Migrations
119. Environments
120. Seed Data
121. Testing Strategy
122. Security Tests
123. Accessibility Tests
124. Performance
125. Large Account Handling
126. Database Indexing
127. Caching
128. Background Jobs
129. Observability
130. Error Monitoring
131. Health Checks
132. Integration Health
133. Backups
134. Deployment
135. Rollback
136. UI Component System
137. Loading States

44. Calendar
45. Insights
46. Completion Forecasts
47. Seasons
48. Year Review
49. Character Profile
50. Public Profile
51. Share Cards
52. Friends
53. Challenges
54. Search
55. Command Palette
56. Focus Sessions
57. Journal
58. Notes
59. Reminders
60. Smart Inactivity Handling
61. Imports
62. Integrations Framework
63. Integration Candidates
64. Deduplication
65. Goal Templates
66. Template Marketplace Architecture
67. Skill Templates
68. User-Created Skills
69. Skill Merge
70. Undo
71. Progress Events
72. Numeric Goal Logic
73. Frequency Goal Logic
74. Duration Goals
75. Open-Ended Goals
76. Binary Goals
77. Goal Progress Calculation
78. Notifications Centre
79. Accessibility
80. Motion
81. Visual Design Direction
82. Dark and Light Themes
83. Responsive Design
84. Offline/PWA Behaviour
85. Data Model
86. IDs
87. Time
88. Data Isolation
89. Backend Architecture
90. Code Architecture
91. Service Layer

138. Empty States
139. Error Recovery
140. Unsaved Changes
141. Deletion Behaviour
142. Undo History
143. Internationalisation Architecture
144. Currency
145. Units
146. Timezones
147. Gamification Settings
148. Celebration Rules
149. Achievement Notifications
150. Daily Summary
151. Weekly Summary
152. User Goal Limits
153. Product Search Engine / SEO
154. Marketing Site
155. Landing Page Messaging
156. Demo Mode
157. Referral System
158. Cosmetic Customisation
159. No Pay-to-Win
160. Data Integrity Invariants
161. Recommended Database Constraints
162. API Design
163. API Versioning
164. Public API Architecture
165. Webhooks
166. Keyboard UX
167. Mobile Gestures
168. Deep Links
169. URL State
170. Archive
171. Goal Relationships
172. Tagging
173. Goal Priorities
174. Deadline Behaviour
175. Progress Confidence
176. Suggested Planning
177. AI Architecture
178. Deterministic Recommendation Engine
179. Support
180. Product Terminology
181. Key Empty-Account Experience
182. Core Daily Loop
183. Core Long-Term Loop
184. Critical UX Rule
185. Performance UX

92. Transaction Boundaries

93. Idempotency

94. Concurrency

186. Production Readiness Gate

187. Codex Build Instruction

188. Final Product Experience

0. Product Definition

Product name: SkillTree IRL

Product type: Consumer/prosumer web application with installable PWA support

Core concept: A persistent RPG-style representation of a person's real-world development.

SkillTree IRL converts real goals, habits, projects, learning, achievements, and activities into a persistent life progression system.

The application must not behave like a gamified todo list.

The core product model is:

Goals change. Skills persist. History persists. The user's character continuously develops.

A user may:

- have many unrelated goals simultaneously
- change their priorities
- pause goals
- abandon goals
- return to previous goals
- change the target of a goal
- create entirely custom goals
- track a skill without having a specific goal
- create projects with many subgoals
- track habits
- record activities that are not part of any goal
- earn achievements
- maintain years of history
- view their development as a persistent skill tree

The application must be usable by someone who wants something simple such as:

Exercise three times per week.

And equally usable by someone managing:

- a business goal
- financial targets
- fitness
- learning
- multiple projects
- long-term personal development

The interface must progressively expose complexity rather than forcing advanced concepts on new users.

1. Core Product Principles

1.1 Permanent character, temporary objectives

The user's SkillTree profile is permanent.

Goals are temporary objectives layered on top of that profile.

Example:

A user works toward:

Launch a software company.

During that process they earn experience in:

- software development
- databases
- design
- marketing
- sales
- business

If the business goal is later abandoned, the experience earned remains.

Never remove legitimate historical progress merely because a goal was changed, paused or abandoned.

1.2 Flexible enough for arbitrary goals

Do not hard-code the application around:

- fitness
- money
- coding
- productivity
- studying

These are templates only.

The system must support arbitrary user-defined goals.

Examples:

- restore a classic car
- read 50 books
- save £20,000
- move country
- learn Japanese
- become a better cook
- launch a company
- complete a degree
- build a house
- improve at chess
- run a marathon
- travel through Europe
- learn piano
- improve relationships
- reduce personal debt
- create an album

The goal engine must be generic.

1.3 Progress without punishment

Avoid mechanics that make users feel that previous effort has been erased.

A user who stops exercising for two weeks should not see:

YOU FAILED.

Instead show:

- current momentum
- previous momentum

- total sessions
- lifetime XP
- personal bests
- progress history

Streaks may exist but must be secondary.

1.4 Real actions create progression

XP should represent meaningful effort.

Do not allow the product to become:

press buttons -> gain meaningless XP.

Every XP transaction should originate from:

- completing a quest
- logging an activity
- reaching a measurable milestone
- completing a habit occurrence
- unlocking an achievement
- verified external activity
- manual correction

XP must be ledger based.

2. Product Navigation

Desktop

Primary left navigation:

- Today
- Goals
- Skills
- Quests
- Habits
- Achievements
- History
- Insights

Secondary area:

- Season
- Profile
- Settings

Persistent primary action:

+ **Add**

The Add action opens a global creation menu.

Options:

- Log activity
- Add progress
- Complete quest
- Create goal

- Create quest
- Create habit
- Add achievement
- Create skill
- Add journal note

Navigation must remain compact.

Avoid giant sidebars and unnecessary nested settings.

3. Mobile Navigation

Bottom navigation:

Today | **Goals** | + | **Skills** | **You**

Tapping + opens a bottom sheet containing:

- Log activity
- Update goal
- Complete quest
- Create goal
- Add habit
- Create quest
- Add note

The **You** screen contains:

- profile
- achievements
- history
- insights
- settings

Pages that would otherwise require desktop-style secondary navigation should use:

- tabs
- segmented controls
- bottom sheets
- expandable sections

Do not create tiny desktop tables on mobile.

4. Authentication

Support:

- email + password
- passwordless email magic link
- Google sign-in
- Apple sign-in where supported
- account recovery
- email verification
- session management
- sign out all sessions

Optional MFA:

- TOTP authenticator application

Security-sensitive actions require recent authentication.

Examples:

- changing email
 - deleting account
 - exporting all personal information
 - removing MFA
-

5. New User Onboarding

Onboarding must take approximately one to three minutes for a user who chooses the default path.

Do not begin with a long questionnaire.

Screen 1 - Welcome

Headline:

Turn real progress into your personal SkillTree.

Primary action:

Build my SkillTree

Secondary:

Explore first

6. Initial Goal Selection

Ask:

What are you currently working toward?

Categories:

- Health & Fitness
- Money
- Career
- Learning
- Business
- Creative
- Personal
- Relationships
- Practical Skills
- Something Else

Allow multiple selections.

The user does not have to select all areas relevant to their life.

7. Suggested Goal Templates

After category selection show common goals.

Example Money:

- Save a specific amount
- Pay off debt
- Build an emergency fund
- Increase income
- Invest consistently
- Track spending better
- Custom goal

Fitness:

- Exercise consistently
- Run 5K
- Run 10K
- Run a marathon
- Gain strength
- Lose weight
- Improve endurance
- Custom goal

Business:

- Start a business
- Launch a product
- Make first sale
- Reach revenue target
- Build an audience
- Custom goal

User may select multiple goals.

Do not require a template.

8. Goal Setup Wizard

The wizard changes depending on goal type.

Basic questions:

What do you want to achieve?

How should progress be measured?

Supported measurement models:

1. Numeric target
2. Currency target
3. Percentage
4. Frequency
5. Duration
6. Milestones
7. Binary completion
8. Recurring consistency
9. Open-ended development
10. Composite goal
11. User-defined measurement

Example:

Goal:

Save £10,000.

Model:

Currency.

Starting value:

£2,300.

Target:

£10,000.

Optional deadline.

The wizard should automatically identify the likely measurement model but allow user correction.

9. Goal Creation - Advanced Configuration

Advanced controls are collapsed by default.

User may configure:

- title
- description
- category
- icon
- target
- unit
- starting value
- deadline
- priority
- visibility
- parent goal
- subgoals
- linked skills
- milestones
- reminder settings
- recurring review
- XP rewards
- evidence expectations
- notes
- tags

The user should never be required to understand XP configuration.

Default XP values are generated automatically.

10. Goal States

Every goal has exactly one lifecycle state:

- Draft
- Active
- Focus
- Later
- Paused

- Completed
- Abandoned
- Archived

Focus means actively prioritised.

Maximum default Focus goals:

3.

User may increase this limit in settings, but the interface should warn that excessive focus goals reduce usefulness.

11. Changing Goals

Goal history must never be destructively overwritten.

If the user changes:

Save £10,000

to:

Save £20,000

record a Goal Revision.

Store:

- previous target
- new target
- date
- optional reason

Display historical progress using the target that was valid at that time while displaying current progress against the new target.

For radical changes provide:

Convert into new goal

This:

12. archives or pauses the original
13. creates a new goal
14. optionally links the new goal as successor
15. carries no goal progress automatically
16. retains skill XP already earned

12. Goal Detail Page

Header contains:

- goal title
- status
- priority
- current progress
- target
- deadline
- contextual actions

Main tabs:

Overview | Plan | Activity | Insights

Overview:

- progress visual
- next recommended action
- current quests
- milestones
- subgoals
- linked skills
- recent activity
- momentum
- notes

Plan:

- milestones
- subgoals
- dependencies
- quests
- recurring tasks

Activity:

- chronological goal history

Insights:

- progress velocity
 - estimated completion
 - consistency
 - strongest contributing skill
 - blockers
 - historical progress
-

13. Composite Goals

Composite goals consist of multiple measurable components.

Example:

Become financially secure

Subcomponents:

- build £10k emergency fund
- clear consumer debt
- maintain three months positive cash flow
- invest monthly for six consecutive months

Each subgoal may have its own measurement method.

Parent progress can be calculated by:

- equal weighting
- manual weighting
- milestone weighting

User may override weights.

14. Goal Dependencies

Goals may depend on other goals.

Example:

Buy a house

depends on:

- save deposit
- improve affordability
- obtain mortgage agreement
- identify property

Dependencies display visually.

Allow:

- blocked
- available
- optional dependency

Never prevent users from manually progressing a goal just because a dependency exists.

Dependencies are planning guidance, not rigid workflow locks.

15. Skills

Skills represent persistent user development.

Examples:

- Running
- Strength Training
- JavaScript
- Public Speaking
- Cooking
- Sales
- Budgeting
- Piano

Skills may exist without goals.

Each skill stores:

- name
- description
- category
- parent skill
- children
- lifetime XP
- level
- discovered date
- recent activity
- evidence strength
- related goals
- achievements
- statistics

16. Skill Taxonomy

The platform has broad top-level domains:

- Health
- Fitness
- Knowledge
- Career
- Business
- Finance
- Creative
- Social
- Practical
- Technology
- Personal Development
- Other

These categories are organisational defaults, not restrictions.

Users may:

- rename custom skills
- create new skill branches
- move skills
- merge duplicate custom skills
- hide skills
- archive skills

System-created canonical skills cannot be deleted globally.

17. Dynamic Skill Discovery

Do not show every possible skill immediately.

Skills should become visible when:

- user selects them
- a goal links to them
- an activity earns XP for them
- user creates them
- system suggests them and user accepts

Example:

User begins website development.

Tree:

Technology

-> Software Development

-> Frontend Development

Later they start PostgreSQL.

Databases becomes visible.

The tree should visually feel like the user's character is expanding.

18. Skill Tree View

The Skill Tree is one of the signature product experiences.

Desktop:

Interactive canvas.

Features:

- pan
- zoom
- centre current branch
- collapse branch
- expand branch
- search
- filter
- highlight recent XP
- highlight skills relevant to active goals
- switch layout

Supported layouts:

- Tree
- Constellation
- Compact hierarchy

Mobile:

Do not attempt unrestricted canvas interaction as the primary view.

Use:

- vertical branching hierarchy
 - expandable nodes
 - optional full-screen interactive tree
-

19. Skill Node

Node displays:

- skill name
- level
- XP progress to next level
- recent activity indicator
- active goal indicator

Clicking opens skill detail.

20. Skill Detail

Displays:

- current level
- lifetime XP
- XP to next level
- recent XP
- lifetime activities

- linked active goals
- achievements
- statistics
- timeline
- parent skill
- child skills

Also:

How you gained this XP

Breakdown:

- quests
- habits
- activities
- milestones
- achievements
- integrations

21. XP System

XP is stored as an immutable transaction ledger.

Never store only a mutable XP total.

Table:

`xp_transactions`

Fields:

- id
- user_id
- skill_id
- source_type
- source_id
- amount
- reason
- confidence_level
- created_at
- reversal_of
- metadata

Source types:

- quest
- activity
- habit
- achievement
- milestone
- integration
- adjustment
- import

Totals are derived or cached safely.

22. XP Difficulty

Default base rewards:

Tiny action: 5 XP

Small action: 10 XP

Moderate action: 25 XP

Significant action: 50 XP

Major action: 100 XP

Milestone: 150-300 XP

Major achievement: 300-1000 XP

Actual XP should account for:

- effort
- difficulty
- duration
- novelty
- recurrence
- verification
- user level

The purpose is consistency, not scientific measurement.

23. XP Diminishing Repetition

Prevent XP farming.

Repeated identical low-difficulty actions gradually reduce bonus XP.

Example:

First 5K:

Large milestone reward.

Normal future 5Ks:

Normal running activity reward.

Repeatedly completing:

Drink water

must not become the easiest route to Level 100.

Use diminishing novelty bonus, not punitive negative XP.

24. Level Formula

Levels must require progressively more XP.

Requirements:

- first levels arrive quickly
- progression visibly slows over time
- no practical maximum level
- deterministic formula

- server authoritative

Example conceptual formula:

```
required_total_xp(level) = base * level^1.65
```

Exact values must be defined in shared server configuration and covered by tests.

Do not duplicate level formulas in clients.

25. Overall User Level

User receives an overall Character Level.

This is calculated from lifetime eligible XP across skills.

Avoid summing every skill's level.

Use total eligible XP.

Display:

- character level
- total XP
- XP this season
- highest skills

Character level is cosmetic and motivational.

It must not unlock essential product functionality.

26. Activities

Activities are the fundamental record of real-world effort.

Examples:

- Ran 4 km
- Studied Spanish for 45 minutes
- Built authentication flow
- Saved £100
- Practised piano
- Completed client presentation
- Cooked new recipe

Activities may be:

- attached to a goal
 - attached to several goals
 - attached to skills
 - attached to no goal
 - generated from a quest
 - generated from a habit
 - imported
-

27. Activity Logging

Global Add -> Log activity.

Form:

- activity description
- optional duration
- optional quantity
- optional unit
- date/time
- related goal
- related skills
- effort
- optional evidence
- private note

The system proposes XP.

User may view:

This activity will earn approximately 25 XP in Running.

Advanced users may adjust skill allocation where permitted.

28. Activity Evidence

Evidence is optional unless the user explicitly configured a quest requiring it.

Evidence types:

- photo
- image
- document
- URL
- text note
- timer record
- external integration reference

Evidence privacy defaults to Private.

Never make evidence publicly visible automatically.

29. Evidence Confidence

Each activity has a confidence classification:

Self reported

User manually recorded it.

Evidence attached

Supporting evidence exists.

Integration verified

Imported from supported external source.

This affects:

- trust indicator
- certain leaderboard eligibility

It should have only a small or zero influence on ordinary XP.

Do not make self-reported users feel second class.

30. Quests

Quests are concrete actions.

Examples:

- Run 3 km
- Deploy landing page
- Read chapter 4
- Save £50
- Contact five potential customers

Quest properties:

- title
 - description
 - related goal
 - linked skills
 - XP
 - due date
 - priority
 - estimated effort
 - recurrence
 - evidence requirement
 - dependencies
 - subtasks
-

31. Quest States

- Planned
- Ready
- In Progress
- Completed
- Skipped
- Cancelled
- Overdue

Completion creates:

- activity event
- progress update where applicable
- XP transaction
- timeline entry

The operation must be transactional and idempotent.

Double clicking cannot award duplicate XP.

32. Suggested Quests

The product may recommend next steps based on templates and current progress.

Example:

Goal:

Run 5K.

Suggested next quest:

Complete a comfortable 3 km run.

Users may:

- accept
- edit
- dismiss

Recommendations are assistance only.

Never automatically insert excessive quests.

33. Habits

Habits represent recurring behaviour.

Examples:

- read 20 minutes
- exercise
- practise guitar
- review spending
- take medication is NOT an intended medical adherence feature

Habit configuration:

- name
 - frequency
 - preferred days
 - minimum target
 - linked goal
 - linked skills
 - XP
 - reminder
 - start date
 - optional end date
-

34. Habit Tracking

Today screen displays due habits.

Actions:

- Complete
- Partial
- Skip
- Add details

Store every habit occurrence separately.

Never overwrite historic completion.

35. Momentum

Momentum is preferred over streak-based pressure.

Momentum should represent recent consistency.

Example:

Running Momentum

High

6 activities in previous 14 days.

Momentum states:

- Building
- High
- Steady
- Cooling
- Inactive

Calculate from configurable rolling windows.

36. Streaks

Streaks are available but secondary.

Display:

- current streak
- best streak

Breaking a streak must not:

- remove XP
 - reset skill progression
 - create negative messaging
-

37. Milestones

Milestones represent significant checkpoints.

Example:

Goal:

Save £10,000.

Milestones:

- £1,000
- £2,500
- £5,000
- £7,500
- £10,000

Milestones can be:

- numeric threshold
- manually completed
- linked to quest completion

- linked to subgoal completion

Milestones can award XP and achievements.

38. Achievements

Achievements are persistent.

Types:

System achievements

Globally defined.

Examples:

- first goal created
- first completed goal
- 100 activities
- Level 10
- 30-day consistent period

Skill achievements

Example:

Running:

- First 5K
- First 10K
- 100 km lifetime

Goal achievements

Generated from goal milestones.

Custom achievements

User created.

39. Achievement Rarity

For platform-defined achievements show aggregate rarity.

Example:

First Marathon

Unlocked by 2.4% of eligible users.

Do not expose exact user lists.

Custom private achievements do not have global rarity.

40. Today Screen

Today is the main operational surface.

Top:

- greeting
- character level

- XP toward next level
- season progress

Then:

Focus

Show maximum three Focus goals.

Each:

- title
- progress
- target/deadline
- next action

Today

Show:

- quests due
- habits due
- scheduled goal reviews

Continue

Surface relevant unfinished activity.

Recent Progress

Show approximately last three meaningful events.

Do not overwhelm the home page with analytics.

41. Next Best Action

Every Focus goal may have a Next Best Action.

Priority order:

17. user-pinned quest
18. overdue high-priority quest
19. nearest dependency
20. upcoming milestone requirement
21. recommended quest
22. prompt to plan next action

Today may show one global:

Recommended next action

Never prevent the user choosing something else.

42. Goal Reviews

Users can schedule:

- weekly review
- monthly review
- custom review

Review flow:

How is this goal going?

Show:

- progress since last review
- completed quests
- XP earned
- milestone movement

Ask optionally:

- Continue?
- Change target?
- Change priority?
- Pause?
- Update deadline?

The review should take less than a minute.

43. History

History is a unified chronological timeline.

Events:

- goal created
- goal changed
- goal paused
- activity
- quest completed
- milestone
- achievement
- skill level-up
- habit record
- imported event

Filters:

- goals
- skills
- activities
- achievements
- dates
- source

Timeline must support years of data efficiently.

Use cursor pagination.

44. Calendar

Calendar views:

- Month
- Week
- Day

Shows:

- activity
- quests
- habit occurrences
- milestone dates
- deadlines

Calendar is not the primary application interface.

It exists for users who prefer date-based planning.

45. Insights

Insights provide useful interpretation rather than meaningless graphs.

Sections:

Momentum

Which areas are currently active?

Progress velocity

Example:

You've averaged £420/month toward this goal over the previous three months.

Consistency

Example:

You completed 79% of scheduled workouts this month.

Skill growth

Largest XP growth during selected period.

Goal health

- On track
- Ahead
- Behind
- No recent activity
- No deadline

Focus distribution

Example:

61% of recent effort has gone toward Business.

Do not imply scientific psychological conclusions.

46. Completion Forecasts

For numeric goals with sufficient historical data:

Estimate completion date based on:

- recent average

- longer-term average
- current target
- remaining progress

Display range rather than false precision.

Example:

At your recent pace: approximately December 2026-January 2027.

Explain calculation.

47. Seasons

Seasons provide temporary progression periods.

Default:

Quarterly seasons.

Example:

Autumn 2026

Season tracks:

- season XP
- activities
- goals completed
- quests completed
- achievements
- strongest skill

At season end generate a Season Summary.

Skill XP and levels never reset.

48. Year Review

Automatically produce:

Your 2026 SkillTree

Contains:

- XP earned
- levels gained
- skills discovered
- biggest skill growth
- goals completed
- goals changed
- major achievements
- most active month
- consistency
- activity count
- personal highlights

Must be visually shareable while excluding private notes/evidence.

49. Character Profile

Profile presents the user as a real-world RPG character.

Contains:

- display name
- avatar
- character level
- lifetime XP
- current season level/progress
- top skills
- active Focus goals
- achievement showcase
- joined date

Allow users to choose which elements appear publicly.

50. Public Profile

Public profile is disabled by default.

Users may enable:

- public
- unlisted
- private

Granular visibility:

- character level
- skills
- specific goals
- achievements
- season stats

Never expose:

- evidence
 - journal entries
 - private notes
 - exact financial values unless explicitly enabled
 - activity location metadata
-

51. Share Cards

Generate social-friendly cards.

Examples:

Level up

Software Development - Level 20

Achievement

First 10K completed.

Season

4,280 XP · 83 activities · 3 completed goals

SkillTree

Visual snapshot.

Allow downloadable:

- portrait
- square
- landscape

Use server-rendered image generation or safe client-side export.

52. Friends

Optional social layer.

Users may follow mutually or connect as friends depending privacy model.

Features:

- profile view
- achievement reactions
- selected progress updates
- optional challenges

Do not build an engagement-feed-first social network.

The user's personal progression remains the main product.

53. Challenges

Users may create:

- personal challenges
- friend challenges

Examples:

- run 50 km this month
- read 4 books
- 20 coding sessions

Challenge scoring must define measurement explicitly.

No cash prizes or gambling mechanics.

54. Search

Global search must locate:

- goal
- skill
- quest
- habit
- achievement
- activity
- journal entry

Keyboard shortcut desktop:

Ctrl/Cmd + K

Search supports fuzzy matching.

55. Command Palette

Desktop power feature.

Commands:

- Create goal
- Log activity
- Find goal
- Find skill
- Complete quest
- Pause goal
- Open settings
- Start focus session

Must not replace normal visible navigation.

56. Focus Sessions

Users can start a timed work session associated with:

- goal
- quest
- skill

Timer:

- start
- pause
- resume
- finish
- discard

On finish:

Create activity from duration.

User can add:

- description
- result
- evidence

Timer state must survive page navigation.

Persist server-side periodically or in recoverable client state.

57. Journal

Optional private journal.

Entries may relate to:

- goal
- activity
- skill
- general day

Support plain rich-text basics:

- paragraphs
- headings
- lists
- emphasis
- links

Do not build a full Notion replacement.

58. Notes

Goal, skill and activity objects may have lightweight notes.

Notes are separate from full journal entries.

59. Reminders

Support:

- email
- web push where available
- in-app

Reminder types:

- quest due
- habit due
- goal review
- inactivity reminder
- deadline approaching

Users control:

- channels
- quiet hours
- frequency
- individual reminder disable

Do not spam.

60. Smart Inactivity Handling

If a user stops interacting:

Do not send:

You're failing.

Use:

Want to continue, pause, or change this goal?

After extended inactivity offer:

- resume
 - pause
 - archive
 - change
-

61. Imports

Support structured imports.

Initial production-supported import formats:

- CSV activity import
- CSV financial progress import
- generic historical activity CSV

Import workflow:

23. upload
24. preview
25. map columns
26. validate
27. show changes
28. confirm
29. process idempotently

Never silently generate duplicate XP from repeated imports.

62. Integrations Framework

Build a generic integration architecture even where provider support expands individually.

Provider table.

OAuth token storage encrypted.

Integration events stored separately before conversion into activities.

Architecture:

External event

-> integration event

-> normalization

-> activity

-> goal progress

-> XP transactions

Provider outages must not break the core app.

63. Integration Candidates

Architecture must support providers such as:

- GitHub
- Strava

- Google Fit / Health Connect where practical
- Apple Health imports where practical
- learning platforms
- financial CSV/import sources
- calendars

Do not require integrations for normal usage.

64. Deduplication

Imported events require provider-specific external IDs.

Unique constraint:

`user_id + provider + external_event_id`

Processing must be idempotent.

65. Goal Templates

Goal templates define:

- title suggestions
- measurement model
- suggested milestones
- suggested skills
- suggested quests
- typical evidence
- useful insights

Templates should never trap user data in rigid domain-specific structures.

66. Template Marketplace Architecture

The production data model should permit curated templates and user-created shared templates.

Shared templates may include:

- goal structure
- milestones
- quests
- linked generic skills

Never include another user's personal information.

Templates require moderation/report controls if publicly searchable.

67. Skill Templates

Similar reusable structures may define:

Learn Web Development

Branches:

- HTML

- CSS
- JavaScript
- Frontend
- Backend
- Databases
- Deployment

Users may accept:

- full tree
 - selected branches
-

68. User-Created Skills

When creating skill ask:

- name
- parent
- description
- icon
- visibility

Prevent obvious accidental duplicates:

"Java Script" looks similar to existing "JavaScript". Use existing skill?

User can ignore suggestion.

69. Skill Merge

Users may merge two personal skills.

Example:

JS + JavaScript

Merge process:

- choose retained skill
- move activities
- move goal links
- move XP transactions
- archive duplicate
- retain merge audit event

Must not double XP.

70. Undo

Important user actions should support undo where safe.

Examples:

- quest completed
- activity logged
- progress updated
- goal paused

Undo XP using reversal transactions.

Never delete financial or XP audit history silently.

71. Progress Events

Goal progress must also be ledger/event based.

Do not only store:

```
current_value = 5000
```

Store:

- progress event
- amount
- resulting value
- source
- date
- user
- optional correction reference

This allows historical charts and corrections.

72. Numeric Goal Logic

Two progress modes:

Absolute update

My savings are now £5,000.

Delta update

I saved another £200.

Clearly distinguish the two.

Prevent common accidental doubling.

73. Frequency Goal Logic

Example:

Exercise 3 times weekly.

Store occurrences.

Progress derives from occurrences within the configured week boundary.

User can configure week starts:

- Monday
- Sunday

Use locale default initially.

74. Duration Goals

Example:

Study Spanish for 100 hours.

Activities contribute duration.

Manual progress corrections permitted.

75. Open-Ended Goals

Example:

Improve drawing.

No completion percentage required.

Display:

- skill growth
- activity frequency
- milestones
- momentum

User may later convert to measurable target.

76. Binary Goals

Example:

Pass driving test.

The goal itself is completed/not completed but may contain:

- milestones
- quests
- subgoals

Do not display fake percentages unless user has milestones configured.

77. Goal Progress Calculation

Goal progress calculators must live server-side.

Define explicit strategy per measurement type.

Never duplicate business rules in React components.

78. Notifications Centre

In-app notification centre supports:

- level up
- achievement
- reminder
- goal deadline

- milestone reached
- friend interaction
- system notice

Support:

- read
 - unread
 - mark all read
 - notification settings
-

79. Accessibility

Target WCAG 2.2 AA.

Requirements:

- full keyboard navigation
- proper semantic elements
- visible focus
- screen reader labels
- accessible tree alternatives
- sufficient contrast
- no information communicated by colour alone
- reduced motion support
- scalable text
- touch targets suitable for mobile

Interactive SkillTree needs an accessible hierarchical list alternative.

80. Motion

Use restrained animation.

Allowed:

- level progress
- achievement unlock
- tree branch appearance
- state transitions

Respect:

`prefers-reduced-motion`

Never make core navigation depend on animation.

81. Visual Design Direction

The product should feel:

- modern
- focused
- slightly game-like
- mature
- premium

- clean

Avoid:

- childish RPG styling
- excessive neon
- pixel-art everywhere
- casino visuals
- giant gradients
- cartoon badges on every surface

Game mechanics should appear through:

- levels
- progression
- skill nodes
- quests
- achievements
- character sheet

The UI itself should remain professional.

82. Dark and Light Themes

Support:

- system
- light
- dark

Theme preference stored per user.

No screen should only be designed for dark mode.

83. Responsive Design

Required production widths:

320 px and above.

Test:

- small mobile
- large mobile
- tablet portrait
- tablet landscape
- laptop
- desktop
- large desktop

Desktop and mobile may use different interaction patterns.

Do not merely shrink desktop.

84. Offline/PWA Behaviour

Installable PWA.

Cache application shell.

Allow limited offline functionality:

- view recent goals
- view recent quests
- create temporary activity
- complete temporary quest

Queued changes sync when online.

Server remains authoritative.

Conflicts must be resolved safely.

Offline XP displayed as:

Pending sync

until server confirms.

85. Data Model

Primary tables:

users

profiles

user_preferences

goals

goal_revisions

goal_dependencies

goal_progress_events

goal_skill_links

goal_milestones

goal_reviews

skills

skill_definitions

user_skills

skill_relationships

quests

quest_goal_links

quest_skill_rewards

quest_dependencies

quest_completions

habits

habit_occurrences

activities

activity_goal_links

activity_skill_links

activity_evidence

xp_transactions

achievements

achievement_definitions

achievement_unlocks

seasons
season_user_stats
journal_entries
integrations
integration_events
notifications
friendships
challenges
challenge_members
subscriptions
entitlements
audit_events
imports
import_rows
user_sessions_metadata

86. IDs

Use UUID or ULID consistently.

Do not expose sequential database IDs.

87. Time

Store timestamps in UTC.

Store user timezone separately.

Render in local timezone.

Recurring habits and deadlines must respect user timezone.

Daylight-saving transitions require tests.

88. Data Isolation

Every user-owned record must be securely scoped to owner.

Backend authorization must not trust:

- client-provided user_id
- hidden form fields
- route ownership assumptions

Use authenticated user identity server-side.

Where Supabase is used:

Enable Row Level Security on every user-owned table.

No table with personal data may rely solely on application filtering.

89. Backend Architecture

Recommended:

Frontend

Next.js with TypeScript.

Backend

Next.js server routes/server actions plus dedicated service modules.

Database/Auth/Storage

Supabase/PostgreSQL.

Payments

Stripe.

Hosting

Cloudflare-compatible or Vercel-compatible deployment depending framework requirements.

The application should remain portable.

Do not place critical domain logic inside provider-specific edge functions unless necessary.

90. Code Architecture

Use domain-oriented project structure.

Example:

```
src/domains/goals
src/domains/skills
src/domains/xp
src/domains/quests
src/domains/habits
src/domains/achievements
src/domains/activities
src/domains/subscriptions
```

Each domain contains:

- types
- validators
- services
- repositories
- policies
- tests

React components should not perform raw SQL or domain calculations.

91. Service Layer

Business rules must exist in testable services.

Examples:

```
completeQuest()
recordGoalProgress()
```

```
awardXp()  
reverseXp()  
changeGoalTarget()  
completeGoal()  
recordActivity()  
processHabitOccurrence()  
unlockAchievement()  
calculateSkillLevel()
```

92. Transaction Boundaries

Operations affecting several records must use DB transactions.

Example completing quest:

30. lock quest/completion condition
31. create completion
32. create activity
33. update goal progress if appropriate
34. generate XP transactions
35. unlock relevant milestones
36. check achievements
37. generate notifications
38. commit

If any required step fails:

Rollback.

93. Idempotency

Mutation endpoints that can be retried require idempotency support.

Examples:

- complete quest
- record imported event
- process Stripe webhook
- award XP
- sync integration
- process offline queue

Use idempotency keys where appropriate plus DB unique constraints.

94. Concurrency

Protect against:

- two devices completing same quest
- duplicate progress updates
- duplicate import
- repeated webhook
- double subscription processing

Use:

- transactions
 - unique constraints
 - row locks where appropriate
 - optimistic concurrency versions where useful
-

95. Validation

All external input validated server-side.

Use shared schemas, e.g. Zod.

Validate:

- lengths
- numbers
- units
- dates
- enumerations
- IDs
- ownership
- files

Never trust client validation alone.

96. Error Model

Return safe structured errors.

Example:

```
{
  "code": "GOAL_NOT_FOUND",
  "message": "This goal could not be found.",
  "requestId": "..."}
}
```

Do not expose:

- SQL
- stack traces
- secrets
- provider credentials

Development logs may contain full diagnostic context.

97. Optimistic UI

Use optimistic interactions for fast actions such as:

- habit completion
- quest completion
- reaction

But rollback visibly if backend fails.

For financially meaningful progress values or XP awards, server response remains authoritative.

98. File Storage

Evidence uploads use private object storage.

Path must be derived server-side.

Never trust user-provided bucket path.

Use signed URLs with short expiry.

Validate:

- MIME type
- extension
- maximum size

Optionally scan uploaded documents where practical.

99. Evidence Limits

Plan-dependent storage limits may exist.

Users must see:

- current storage
- allowed storage

Do not silently delete evidence when subscription changes.

Instead place account over limit and restrict additional uploads if necessary.

100. Subscription Model

Suggested plans:

Free

- unlimited core skill tracking
- limited active goals
- basic quests
- basic habits
- standard SkillTree
- limited evidence storage
- basic insights

Pro

Approximately £5-£10/month or annual equivalent.

Includes:

- unlimited active goals
- advanced insights
- historical analysis
- full seasons/year reviews
- advanced SkillTree layouts

- custom goal templates
- advanced recurring rules
- expanded evidence
- integrations
- export features
- custom profile/share cards

Avoid crippling the free experience.

Users need enough functionality to become attached to their history.

101. Subscription Billing

Stripe Checkout.

Stripe Customer Portal.

Store subscription state server-side.

Never trust Stripe status from frontend.

Webhook events:

- checkout completed
- subscription created
- subscription updated
- subscription cancelled
- invoice paid
- invoice payment failed

Webhook handling idempotent.

102. Entitlements

Do not scatter:

```
if plan === "pro"
```

through UI.

Create entitlement service.

Examples:

```
canUseAdvancedInsights
```

```
maxActiveGoals
```

```
evidenceStorageLimit
```

```
canCreateCustomTemplates
```

```
canUseIntegration(provider)
```

Backend enforces entitlements.

Frontend mirrors them for UX only.

103. Trial

Optional Pro trial.

Server controls:

- start
- expiry
- entitlement

Do not require card unless product strategy chooses it.

At expiry:

- never delete data
 - safely downgrade
 - preserve historical records
 - disable only features exceeding Free entitlement
-

104. Account Export

Allow complete user export.

Formats:

- JSON master export
- CSV activities
- CSV XP ledger
- CSV goals

Evidence optionally bundled where practical.

Export must be generated securely and expire.

105. Account Deletion

Deletion flow:

39. authenticate recently
 40. explain effect
 41. enter confirmation
 42. optional grace period
 43. revoke sessions
 44. cancel subscription
 45. remove/anonymise personal records as required
 46. delete private evidence
 47. maintain legally required billing records separately
-

106. Privacy

Privacy-first defaults.

Private by default:

- goals
- financial values
- evidence
- journal
- activities
- skills

User deliberately opts into sharing.

107. Analytics

Product analytics must not require storing sensitive goal text where avoidable.

Track events such as:

- signup completed
- goal created
- activity logged
- quest completed
- habit completed
- profile shared
- Pro trial started
- subscription purchased
- subscription cancelled

Do not include private free-text content in analytics event payloads.

108. Product Metrics

Internal metrics:

Activation:

- created first goal
- logged first activity
- earned first XP

Retention:

- WAU
- MAU
- activities/week
- active goals
- returning users

Value:

- goals progressed
- goals completed
- sessions
- skill development

Revenue:

- trial conversion
 - MRR
 - churn
 - annual conversion
-

109. User Feedback

Provide:

Settings -> Feedback.

Categories:

- Bug
- Feature request
- Confusing
- Other

Include optional diagnostic context:

- app version
- route
- device
- browser

Never include private goal data without explicit consent.

110. Admin Platform

Separate admin access from customer application.

Roles:

- support
- admin
- superadmin

Features:

- view user metadata
- subscription status
- account status
- feature flags
- error events
- integration health
- support notes

Admin must not casually expose:

- private journals
- evidence
- goal content

Sensitive content access must be restricted, audited and only implemented where operationally necessary.

111. Feature Flags

Server-controlled feature flags.

Use for:

- rollout
- experiments
- emergency disable
- provider integration disable

Flags may target:

- all users
- percentage
- specific environment

- specific account
-

112. Audit Logging

Audit sensitive events:

- email changed
- password reset initiated
- MFA changed
- account deletion
- public visibility changed
- subscription changed
- admin actions
- entitlement override

Audit events immutable.

113. Rate Limiting

Rate-limit:

- login
- password recovery
- invitation/public link endpoints
- uploads
- search
- public profiles
- API mutations
- integration callbacks

Use user/IP contextual limits.

114. Abuse Controls

Public/shared surfaces need:

- report
- block
- moderation
- rate limits

User-generated public templates require moderation state.

115. Security Headers

Production:

- CSP
- HSTS
- Referrer Policy
- Permissions Policy
- X-Content-Type-Options

- secure cookies
- SameSite cookies where applicable

No secrets bundled client-side.

116. CSRF

Any cookie-authenticated state mutation must have appropriate CSRF protection or framework-provided equivalent.

Test explicitly.

117. XSS

User-generated rich content must be sanitised.

Prefer structured editor output.

Never render arbitrary user HTML.

118. Database Migrations

All DB schema changes through version-controlled migrations.

No manual production schema editing.

Migrations tested against:

- clean DB
 - populated previous-version DB
-

119. Environments

Separate:

- local
- test
- staging
- production

No shared database.

No production credentials locally.

Environment-specific configuration validated at startup.

120. Seed Data

Provide realistic development seed scenarios.

New User

One simple goal.

Multi-goal User

- fitness
- finance
- learning

Long-term User

Years of history.

Power User

Dozens of skills, quests, goals.

Free User

At entitlement limits.

Pro User

All features.

121. Testing Strategy

Unit Tests

Domain rules:

- XP calculation
- level calculation
- goal progress
- milestone unlock
- achievement unlock
- entitlement checks
- recurrence
- momentum

Integration Tests

- database services
- transactions
- authorization
- Stripe webhooks
- integration deduplication

End-to-End Tests

Critical flows:

- registration
- onboarding
- goal creation
- activity logging
- quest completion
- habit completion
- goal edit
- goal pause
- goal completion

- XP award
 - achievement
 - subscription
 - export
 - deletion
-

122. Security Tests

Automated authorization testing:

User A must never:

- read User B goal
- modify User B activity
- access User B evidence
- complete User B quest
- retrieve signed evidence URL
- update User B profile

Test malicious direct API requests, not only UI.

123. Accessibility Tests

Automated:

- axe

Manual:

- keyboard only
 - screen reader smoke testing
 - zoom 200%
 - reduced motion
 - colour-blind information checks
-

124. Performance

Targets:

Common navigation:

perceived instant after initial load.

API:

typical simple reads under \~500ms under normal load.

SkillTree:

must remain usable with hundreds of skills.

History:

cursor pagination.

Do not load entire lifetime activity history at startup.

125. Large Account Handling

Test account with:

- 100 goals
- 500 skills/nodes
- 20,000 activities
- 50,000 XP transactions
- years of history

UI must remain responsive.

Use:

- pagination
 - virtualization where necessary
 - aggregation tables/views
 - indexes
-

126. Database Indexing

Index commonly queried:

- user_id
- created_at
- status
- goal_id
- skill_id
- parent_id
- due_at
- integration external IDs

Review queries with `EXPLAIN ANALYZE`.

127. Caching

Cache only derived/non-sensitive values where useful.

Examples:

- public template library
- public profile
- skill definitions

Never use cache as authorization layer.

128. Background Jobs

Use durable job handling for:

- reminders
- email
- imports
- integration sync
- annual summaries

- image generation
- exports

Jobs need:

- retries
 - exponential backoff
 - dead-letter handling or failed status
 - idempotency
-

129. Observability

Structured logs.

Include:

- request ID
- user ID where safe
- route
- duration
- status
- error code

Never log:

- passwords
 - OAuth tokens
 - private journal text
 - evidence content
-

130. Error Monitoring

Integrate production exception monitoring.

Capture:

- frontend crashes
- API exceptions
- failed jobs
- provider errors

Include environment and release version.

131. Health Checks

Provide:

`/health/live`

`/health/ready`

Readiness checks:

- database connectivity
- critical configuration

Avoid dependency checks that make the app fully unavailable due to optional external providers.

132. Integration Health

Provider health tracked separately.

Example:

GitHub integration degraded.

Core application continues working.

User sees:

GitHub syncing is temporarily delayed.

Not:

Application unavailable.

133. Backups

Production database:

automatic backups.

Document restore process.

Perform periodic restore test.

Evidence storage requires retention/backup strategy consistent with provider capabilities.

134. Deployment

CI pipeline:

48. install locked dependencies

49. lint

50. typecheck

51. unit tests

52. integration tests

53. build

54. migration validation

55. deploy staging

56. smoke test

57. production deploy

Do not deploy failing main branch.

135. Rollback

Application releases must be reversible.

Migrations should prefer backward-compatible patterns.

Destructive migrations use phased migration:

58. add new structure

59. dual compatibility

60. migrate data

- 61. switch code
 - 62. later remove old structure
-

136. UI Component System

Create reusable components:

- Button
- IconButton
- Input
- TextArea
- Select
- Checkbox
- Switch
- Modal
- Drawer
- BottomSheet
- Tabs
- SegmentedControl
- Toast
- EmptyState
- ErrorState
- Skeleton
- ProgressBar
- ProgressRing
- XPBar
- SkillNode
- GoalCard
- QuestRow
- HabitRow
- AchievementBadge
- TimelineItem

No page should invent unrelated control styling.

137. Loading States

Every asynchronous surface requires intentional loading state.

Use skeletons where structure is known.

Never leave blank white page.

138. Empty States

Examples:

No goals:

What do you want to work toward?

Button:

Create your first goal

No activities:

Your progress history will appear here when you start logging activity.

Empty states must teach next action.

139. Error Recovery

Every major page needs:

- human-readable error
- Retry
- safe navigation away

Failed mutations:

- preserve user-entered data
- show reason
- allow retry

Do not clear a long goal form after network failure.

140. Unsaved Changes

Forms with substantial edits:

- autosave drafts where appropriate
- or warn on navigation

Goal creation can save local/server draft.

141. Deletion Behaviour

Prefer Archive for:

- goals
- skills
- templates

Hard delete only when user intentionally requests it and dependencies permit.

Deletion must explain consequences.

142. Undo History

For corrections:

Use reversal events.

Example:

Accidental +£500 goal progress.

Correction:

- create reversing progress event

- reverse related XP if necessary

Preserve auditability.

143. Internationalisation Architecture

Build text through translation keys.

Initial language may be English.

Do not hardcode every UI string inside components.

Support locale-specific:

- dates
 - decimal separators
 - currencies
 - first day of week
-

144. Currency

Goals may use any supported ISO currency.

Currency belongs to the goal, not global account.

One user may track:

- GBP savings
- USD revenue

Do not automatically exchange currencies unless explicit future provider logic is available.

145. Units

Support:

- distance
- weight
- duration
- count
- percentage
- currency
- custom units

Store canonical units where conversion is supported.

146. Timezones

Users can change timezone.

Historical timestamps remain absolute.

Recurrence future schedule recalculates intelligently.

Warn where timezone shift changes due-date interpretation.

147. Gamification Settings

Users can customise intensity.

Options:

Full

XP, levels, celebrations, achievements.

Balanced

Default.

Minimal

Progress tracking with reduced game presentation.

Do not make users choose this during initial onboarding.

148. Celebration Rules

Celebrate meaningful events:

- level up
- goal complete
- rare achievement
- significant milestone

Do not celebrate:

- every checkbox
- trivial actions

Allow celebrations off.

149. Achievement Notifications

Small toast for ordinary achievement.

Larger modal/sheet only for significant achievement.

Never interrupt an ongoing timer unnecessarily.

150. Daily Summary

Optional.

Shows:

- activities
- XP
- completed quests
- habits
- goal movement

No guilt language.

151. Weekly Summary

Optional email/in-app.

Contains:

- XP earned
 - strongest skill
 - goals progressed
 - consistency
 - next week focus
-

152. User Goal Limits

Free plan may limit currently Active/Focus goals.

Archived/completed history must remain unlimited.

Never require deletion of historical goals to fit plan.

153. Product Search Engine / SEO

Public marketing site should include useful pages around:

- goal tracking
- skill development
- real-life RPG
- personal skill tree
- habit progression
- life goals
- progress tracking

Do not expose user private pages to indexing.

154. Marketing Site

Routes:

- Home
- Features
- SkillTree
- Goals
- Pricing
- Examples
- Privacy
- Security
- Terms
- Support
- Sign in
- Start free

Marketing screenshots/demo use synthetic accounts.

155. Landing Page Messaging

Primary concept:

Level up your real life.

Secondary explanation:

Track goals, build skills, complete quests and watch your real-world SkillTree grow over time.

Avoid marketing it as merely:

- productivity software
 - todo manager
 - habit tracker
-

156. Demo Mode

Allow visitors to explore a read-only synthetic SkillTree.

Demo should contain:

- several goals
- skill tree
- achievements
- history

At key interaction:

Create your SkillTree

157. Referral System

Optional referral program supported in production architecture.

Referral link.

Track:

- referral signup
- conversion

Rewards may include:

- cosmetic profile items
- Pro time

Avoid pyramid-style mechanics.

158. Cosmetic Customisation

Allow:

- profile accent
- avatar
- SkillTree style
- achievement showcase
- profile layout

Cosmetics must not reduce accessibility.

159. No Pay-to-Win

Never sell XP.

Never sell levels.

Never allow money to fabricate achievements.

Subscription purchases functionality/customisation, not progression.

160. Data Integrity Invariants

Must always hold:

- XP cannot exist without source/reason.
 - Duplicate quest completion cannot grant duplicate XP.
 - Goal ownership cannot change through client request.
 - Skill XP total equals valid ledger transactions.
 - Corrections use reversals.
 - Subscription entitlement enforced server-side.
 - Archived goals retain activity.
 - Goal deletion never silently deletes XP history.
 - User deletion removes/anonymises owned private content.
 - Integration retries do not duplicate activities.
-

161. Recommended Database Constraints

Use:

- foreign keys
- unique constraints
- checks
- NOT NULL
- enums or validated text as appropriate

Examples:

Unique quest completion where only one completion is permitted.

Unique integration external event.

XP amount bounded within safe limits.

Progress values validated.

162. API Design

Use explicit domain endpoints/actions.

Examples:

POST /api/goals

GET /api/goals/:id

PATCH /api/goals/:id

POST /api/goals/:id/progress
POST /api/goals/:id/pause
POST /api/goals/:id/resume
POST /api/goals/:id/complete
POST /api/activities
POST /api/quests/:id/complete
POST /api/habits/:id/occurrences
GET /api/skills/tree
GET /api/history
Avoid generic:
POST /api/update-record

163. API Versioning

Internal first-party API may start versioned:

/api/v1

Public API, if exposed, must always be versioned.

164. Public API Architecture

Pro accounts may eventually create API keys, but architecture should be present.

Keys:

- hashed at rest
- scoped
- revocable
- last-used timestamp
- rate limited

Scopes:

- read goals
- write activities
- read skills

No unrestricted master token.

165. Webhooks

If external developer API is supported:

User can configure outbound webhooks.

Events:

- goal.completed
- skill.levelled
- achievement.unlocked
- activity.created

Sign payloads.

Retry safely.

Provide delivery log.

166. Keyboard UX

Desktop shortcuts:

C create/log

G goals

S skills

Q quests

/ search

Shortcuts disabled while typing.

Display shortcut reference.

167. Mobile Gestures

Use conventional gestures sparingly.

Do not make essential actions swipe-only.

Swipe may optionally:

- complete quest
- defer quest

Every swipe action must also have visible button/menu equivalent.

168. Deep Links

Every major object has stable route.

Examples:

/goals/:id

/skills/:id

/activities/:id

Links should open correct object after authentication.

169. URL State

Filters/tabs that users may share/bookmark should be represented where reasonable in URL query parameters.

170. Archive

Unified archive allows viewing:

- paused
- abandoned

- archived
- completed

Filter by type/date.

Resume supported where meaningful.

171. Goal Relationships

Support:

- parent
- child
- dependency
- successor
- related

Do not force all relationships into a simple tree.

172. Tagging

Users can optionally add tags.

Example:

work

2026

health

Tags supplement structured relationships.

Do not require tagging.

173. Goal Priorities

Priority:

- Focus
- High
- Normal
- Low
- Later

Do not require numeric priority scoring.

174. Deadline Behaviour

Deadlines may be:

- exact
- target
- none

Missing deadline is acceptable.

The product must not imply every life goal requires a deadline.

175. Progress Confidence

For automated forecasts distinguish:

- enough data
- limited data
- insufficient data

Never manufacture estimates from one data point.

176. Suggested Planning

When creating broad goal such as:

Learn guitar.

System may suggest:

- practise basic chords
- learn first song
- establish weekly practice
- learn scales

The user controls whether suggestions are added.

177. AI Architecture

The core product must work without generative AI.

If an AI-assisted goal planner is implemented:

- treat output as suggestions
- user confirms changes
- never let AI directly alter XP ledger
- never send sensitive private content unnecessarily
- provider abstraction
- strict cost limits
- failure fallback to deterministic templates/manual editing

AI failure must not break the app.

178. Deterministic Recommendation Engine

Provide useful recommendations without AI.

Rules based on:

- goal type
- stalled goal
- overdue quest
- nearest milestone
- user-selected focus
- previous frequency

Example:

You haven't updated this goal for 14 days. Review it?

179. Support

In-app Support:

- search help
- contact support
- report bug
- export diagnostic ID

Help content includes:

- how XP works
 - how goals differ from skills
 - changing goals
 - privacy
 - subscriptions
 - imports
-

180. Product Terminology

Use consistent terminology.

Goal

Outcome being pursued.

Skill

Persistent capability.

Quest

Concrete action.

Activity

Historical action record.

Habit

Recurring behaviour.

Milestone

Checkpoint.

Achievement

Permanent notable accomplishment.

XP

Progression experience.

Season

Temporary progress period.

Never alternate between task/quest randomly.

181. Key Empty-Account Experience

A new user should see:

Focus goal.

One recommended quest.

Visible SkillTree start.

Not:

50 empty dashboard widgets.

182. Core Daily Loop

The normal recurring interaction should be:

63. Open Today.

64. See current Focus goals.

65. See next useful action.

66. Complete/log real activity.

67. Receive concise progress feedback.

68. Observe relevant goal/skill movement.

69. Leave.

This should take seconds when desired.

183. Core Long-Term Loop

Weekly/monthly:

70. Review goals.

71. See what changed.

72. Adjust targets/priorities.

73. Pause irrelevant goals.

74. Create new objectives.

75. View SkillTree growth.

76. Continue.

The product should become more valuable as history grows.

184. Critical UX Rule

Never require the user to maintain the system more than they maintain their real goals.

If entering a 30-minute workout requires five minutes of administration, the product has failed.

Common actions need extremely short flows.

185. Performance UX

After user submits activity:

Do not block interface while computing every secondary achievement.

Immediate server transaction handles essential:

- activity
- progress
- XP

Secondary tasks such as:

- summary aggregation
- certain notifications

may run asynchronously.

186. Production Readiness Gate

Do not consider the application production-ready until all of the following are true:

Authentication works end-to-end.

Authorization is hostile-tested.

RLS is enabled and tested.

Goals support every defined measurement type.

Goal revision/history works.

Skills persist independently of goals.

SkillTree is responsive.

XP ledger and reversal system works.

Quest completion is idempotent.

Habits work across timezone boundaries.

Achievements cannot duplicate.

Activity logging works on mobile and desktop.

Evidence storage is private.

Subscription/entitlements are backend-authoritative.

Stripe webhooks are idempotent.

Account export works.

Account deletion works.

Public sharing defaults private.

Notifications respect preferences.

Offline queued mutations cannot duplicate progress.

Large-account performance tests pass.

Accessibility testing passes.

Critical E2E flows pass.

Error monitoring is active.

Backups exist.

Restore procedure has been tested.

Staging environment exists.

Production deployment has rollback procedure.

Terms/privacy/security/support pages exist.
No critical placeholder behaviour remains.
No mock authentication remains.
No client-side-only permission logic remains.
No hardcoded development secrets remain.

187. Codex Build Instruction

Treat this specification as the source of truth.
Do not intentionally reduce this into an MVP.
Do not leave major defined workflows as placeholders.
Build the architecture so all described product behaviours coexist coherently.
Before implementing a feature, identify:

- domain model
- user workflow
- authorization
- validation
- persistence
- error handling
- loading state
- mobile behaviour
- desktop behaviour
- analytics
- testing

Do not solve defects by adding frontend-only exceptions around incorrect backend behaviour.

Fix domain logic at the authoritative layer.

Use TypeScript strict mode.

No [any](#) without documented justification.

Use migrations for database changes.

Keep application/domain logic separate from presentation.

Use transactions for multi-record state changes.

Test hostile authorization.

Keep all private data private by default.

Every important mutation must be safe under retries.

Every long-running process must fail gracefully.

The final product must feel like one cohesive system rather than a collection of independently implemented features.

188. Final Product Experience

The finished product should make a user's life appear as a continuously developing character.

At any point they can open SkillTree IRL and understand:

What am I currently trying to achieve?

What should I work on next?

What have I accomplished?

What skills have I actually developed?

Where has my time and effort gone?

How have I changed during the last month, year or several years?

A user may completely change direction without losing the value of their previous history.

Example:

2026:

Launch Business

-> paused.

2027:

Learn Cybersecurity

-> completed several milestones.

2028:

Return to Business.

Their profile might now show:

Software Development - Level 34

Cybersecurity - Level 27

Business - Level 22

Marketing - Level 14

Finance - Level 18

The person's goals changed.

Their character did not reset.

That persistent identity and progression system is the defining feature of SkillTree IRL.